

skycell: a cost-based HEALPix index for sky positions in PostgreSQL

Design, brute-force validation, and a controlled comparison with Q3C and pgSphere

J. Salgado¹  and M. Delli Veneri¹ 

SKA Observatory, Jodrell Bank, Lower Withington, Macclesfield SK11 9FT, United Kingdom
jesus.salgado@skao.int, michele.delliveneri@skao.int

Received ...; accepted ...

ABSTRACT

Context. Astronomical archives answer positional queries from relational databases, and in the PostgreSQL world two extensions dominate: Q3C, which keys each source by an integer drawn from a quad-tree on a cube and stores it in a B-tree, and pgSphere, which stores spherical geometry in a GiST tree. The first is compact and fast for points; the second is general and indexes regions as well.

Aims. We ask whether the two can be combined: a single index that keeps the integer key and the B-tree, yet answers cones, polygons, cross-matches and stored footprints, and that speaks the geometry language archives already use. We aim to establish where such an index helps, where it does not, and what its cost model actually depends on.

Methods. We present **skycell**, a PostgreSQL extension that keys positions by their order-29 HEALPix cell in the nested scheme and turns a query region into index ranges. How finely to cut is decided by a cost model – the price of an extra index range against the rows it removes – with the local source density read from the histogram **ANALYZE** already keeps for the key column, which is a density map precisely because HEALPix cells are equal area. Cells are classified conservatively against the region, and a planner support function rewrites the predicate into B-tree range conditions plus an exact test. Coverings are validated by brute force against millions of catalogue points and against points drawn inside adversarial cones and polygons. Performance is measured against Q3C 2.0.5 and pgSphere 1.5.2 on two 10-million-row corpora – one synthetic, one resampled from the real Gaia DR3 density field – in trials where all three methods answer the same query back to back in a randomized order, repeated, analysed paired, warm and with caches dropped, and repeated again on a 50-million-row catalogue where pgSphere’s index no longer fits in memory.

Results. Warm, skycell is faster than pgSphere by 6–11% for cone radii of 1′–1′ and by 31–37% at 3°, and the two are indistinguishable between 6′ and 1°; on the Gaia field skycell is 3% slower at 6′. With caches dropped it is faster at every radius on both corpora (ratios 0.39–0.95), touching a third of pgSphere’s buffer pages at 1°. It keeps an index of 214 MB against pgSphere’s 685 MB – 22.5 bytes per row against 72, which at Gaia DR3’s 1.8×10^9 sources is 38 GiB against the 160 GiB of the pgSphere-plus-Q3C pair an archive needs today to cover both cone searches and cross-matching – builds it in 1.4 s against 42 s, answers convex polygons 2.7–4.4 times faster, matches **q3c_join** on cross-matching – the operation Q3C is built for – while being 2.5–3.3 times faster than pgSphere at it, and its row estimates are wrong by a factor 1.2 rather than 2.5–3.0. The cost model’s one parameter is derived from the relation’s own statistics and the planner’s cost factors rather than configured, its chosen cell order sits about one order coarser than the empirical optimum at a cost of 0.8–2% of query time, and its density estimate is unbiased wherever a density is defined at all. Raising the catalogue to 50 million rows separates the two halves of the result: for regions of 30′ and larger the advantage grows (at 1°, 0.85 to 0.64 warm and 0.53 to 0.27 cold), while for cones of an arcsecond to a few arcminutes it shrinks and reverses, skycell becoming 7–13% slower warm, because its planning cost grows with source density while pgSphere’s does not. An ObsCore-shaped relation measured at three sizes puts the crossover with pgSphere between 0.5 and 2 million rows for degree-scale cones, and beyond 10 million for small cones on field-clustered data.

Conclusions. A cost-based covering over an equal-area integer key is a sound basis for a sky index, and the right question is not whether it is faster but for which queries. Its cost is paid per query and its benefit per row scanned, so it wins where enough rows are scanned to amortise a covering – degree-scale regions, polygons, cross-matching, cold caches – and that boundary moves up in radius as the catalogue grows denser. On a Gaia-sized catalogue a small cone search is not the workload this index improves.

Key words. Astronomical databases: miscellaneous – Catalogs – Virtual observatory tools – Methods: data analysis

1 Introduction

Almost every question asked of an astronomical archive is positional: what is within this cone, which observations cover this field, which of my sources match that catalogue. In the Virtual Observatory

these questions arrive as ADQL (Ortiz et al. 2008) through the Table Access Protocol (Dowler et al. 2019), and the service has to turn them into something a relational database can answer with an index.

Two PostgreSQL extensions carry most of this traffic. Q3C (Koposov & Bartunov 2006) projects

the sphere onto a cube, subdivides each face with a quad-tree, and numbers the cells along a space-filling curve; each source is one 64-bit integer in a B-tree, and the table can be sorted along the same curve so that neighbours on the sky are neighbours on disk. pgSphere (Chilingarian et al. 2004) takes the other route: it adds spherical types – points, circles, polygons, boxes – and indexes them with GiST over bounding volumes, so that regions can be stored and queried as data, not only as query arguments.

The scale this has to work at is set by archives already in operation. The ESA Gaia Archive (Salgado et al. 2017, 2020) serves a five-parameter astrometric catalogue of order 10^9 sources through an extended TAP interface, and its published description puts cross-match – matching a list of positions a user has uploaded against the catalogue – among the server-side capabilities implemented beside the query interface itself, under the principle of moving the code close to the data rather than the data to the user. Cross-match is therefore not a synthetic benchmark operation: it is a service an archive of this kind is expected to provide, and Sect. 6.8 treats it as a first-class result rather than an afterthought.

Neither idea is new to astronomy. The Hierarchical Triangular Mesh (Kunszt et al. 2001) had already keyed positions by a subdivided polyhedron, and the zones algorithm (Gray et al. 2006) showed how far a purely relational arrangement – declination strips and a B-tree – can be pushed for cross-matching. What has changed since is the size of the catalogues and the fact that most queries now arrive through a generated layer (e.g. Demleitner et al. 2014) rather than being written by hand, so the index has to serve a translator’s output rather than a tuned query.

The two designs trade off against each other. A linear key in a B-tree is compact, cheap to probe and orders the heap, but a region has to be turned into a set of key ranges before the index can be used, and a set of ranges is an approximation whose quality depends on how finely the region is cut. A tree of bounding volumes has no such approximation step and handles extended objects naturally, but its keys are larger, its nodes overlap, and it imposes no useful order on the table.

This paper presents `skycell`, which keeps the integer key and asks a cost question at the point where the classical approach fixes a constant: how finely should a region be cut? The answer depends on two quantities the database already knows – how many sources sit in that part of the sky, and how much an extra index range costs – and it can be computed rather than configured.

Because the interesting claims here are quantitative, the measurements are built to survive scrutiny rather than to flatter the method: every query is answered by all three methods back to back in a randomized order, repeated and analysed paired; everything is measured both warm and with the page

Table 1: Notation.

Symbol	Meaning
k	HEALPix order, $0 \leq k \leq 29$
s_k	cell size at order k , $s_0/2^k$ rad
A_k	cell area, $4\pi/(12 \cdot 4^k)$ sr
R	query region (cone or convex polygon)
A_{region}	area of R , sr
r	cone radius, rad
ρ	local source density, rows sr $^{-1}$
c_{range}	price of one index range, in rows
$C(s)$	cost of a covering of cell size s , rows
n_{ranges}	key ranges in the covering
A_{cover}	area covered by the covering, sr
$\text{maxpixrad}(k)$	centre-to-boundary bound, rad

cache dropped; and everything is repeated on a second corpus whose density structure comes from the real Gaia DR3 sky rather than from the authors. Section 2 develops the method, Sect. 3 its integration with a query planner, Sect. 4 what is proven and what is only tested, Sect. 5 the protocol, Sect. 6 the results, and Sect. 7 a TAP service and the measured crossover. Section 8 states plainly what these measurements do not establish.

2 Method

Table 1 collects the notation.

2.1 An equal-area integer key

A position is keyed by the index of the HEALPix cell containing it at order 29 in the nested scheme (Górski et al. 2005), computed by the standard `ang2pix_nest` arithmetic and stored as a signed 64-bit integer. At order 29 there are $12 \cdot 4^{29} \approx 1.4 \times 10^{19}$ cells of about 0.4 mas, so the key needs 63 bits and a PostgreSQL `bigint` holds it without reinterpretation.

The encoding is the plain nested index at a fixed order, not a multi-order key: the order is implicit, so a key is directly comparable with another key and a B-tree orders them along the HEALPix curve. Three properties follow, and only the first is shared with the cube quad-tree of Q3C.

Cells are *nested*: the order-29 descendants of a cell p at order k are exactly the contiguous interval

$$[p \cdot 4^{29-k}, (p+1) \cdot 4^{29-k} - 1], \quad (1)$$

so a region covered by cells of any orders becomes a union of integer intervals and a B-tree can answer it with range scans. The NUNIQ encoding of the MOC standard (Fernique et al. 2014), used here for stored regions (Sect. 2.5), packs the order into the key as $4 \cdot 4^k + p$; the two are related by Eq. (1), and `skycell` converts between them rather than storing two representations.

Cells are *equal area*. Every cell at a given order subtends the same solid angle, so a count of keys in

an interval divided by the interval’s length in cells is proportional to a source density per unit solid angle. The histogram that ANALYZE keeps for the key column is therefore a map of how crowded each part of the sky is (Sect. 2.3), without any extra statistics machinery.

Cells are *standard*. The same numbering underlies Gaia’s `source_id` (Gaia Collaboration 2016) and the IVOA’s Multi-Order Coverage maps (Fernique et al. 2014), so keys, coverings and stored footprints are interoperable rather than private to one extension.

2.2 Coverings decided by cost

A query region is covered by cells whose union contains it; the covering becomes range conditions on the key, and rows inside the covering but outside the region are removed by an exact test. Cutting finer means fewer such false positives and more ranges. With cells of size s covering a region of characteristic radius r and a local source density ρ ,

$$C(s) = n_{\text{ranges}}(s) c_{\text{range}} + \rho (A_{\text{cover}}(s) - A_{\text{region}}). \quad (2)$$

Both terms are in rows: the second is the expected number of false positives, and c_{range} converts one additional index range into the number of false-positive rows it is worth avoiding. It is therefore a property of the installation, not of the sky: the ratio between the cost of opening one more B-tree range scan (a descent of the tree plus a planner path with its own selectivity estimate) and the cost of fetching and discarding one heap row. Section 6.10 measures it.

A covering follows the region’s boundary, so for $s \ll r$ the number of ranges grows as r/s – the curve enters and leaves the region once per boundary crossing, and the number of such clusters is the classical measure of a space-filling curve’s quality (Moon et al. 2001) – while the wasted area grows as the perimeter times s . Writing $A_{\text{cover}} \simeq \pi(r + \beta s)^2$ and $n_{\text{ranges}} \simeq \gamma r/s$ and minimising Eq. (2) gives

$$s_{\star} = \sqrt{\frac{\gamma c_{\text{range}}}{2\pi\beta\rho}}, \quad (3)$$

which does not depend on the size of the region: the density of the sky and the price of a range decide how finely it is worth cutting. For $s \gtrsim r$ the covering is a handful of cells and the balance differs, so rather than solve the branches the implementation evaluates Eq. (2) at all 30 orders – a few floating-point operations each – and takes the cheapest, which also lets a cap on the number of ranges and a guard on cell size simply remove candidates. The constants $\beta = 0.7$ and $\gamma = 2$ are geometric; Sect. 6.10 shows the choice of order is flat enough near its optimum that they do not need to be exact.

Figure 1 shows the two extremes on the same cone radius: at high Galactic latitude the covering is a single range of large cells covering 5.6 times the cone’s

area, because there is almost nothing to exclude; in the bulge the same cone is covered by 23 cells in 8 ranges covering 1.3 times its area, because there the rows removed are worth the extra ranges.

Two guards keep the model safe against its own inputs. A histogram is a coarse synopsis, and a dense cluster split by the space-filling curve into pieces smaller than one bucket can be underestimated; a partly covered cell is therefore never allowed to exceed a multiple (64 by default) of the region’s own area, which bounds the damage to the density-proportional behaviour a fixed-depth covering would have had. Gaps between ranges are merged when the rows they contain cost less than the range they save.

2.3 The density estimate

PostgreSQL’s ANALYZE stores, for the indexed expression, an equi-depth histogram: a sorted list of $B + 1$ bounds $h_0 < h_1 < \dots < h_B$ in key space such that each bucket $[h_i, h_{i+1})$ holds about N/B of the table’s N rows. Because the key is an equal-area cell index, the rows in a key interval $[a, b]$ are estimated by linear interpolation within the buckets it spans,

$$\hat{n}(a, b) = \frac{N}{B} \sum_i \frac{|[a, b] \cap [h_i, h_{i+1})|}{h_{i+1} - h_i}, \quad (4)$$

and the corresponding density in rows per steradian is

$$\hat{\rho}(a, b) = \frac{\hat{n}(a, b)}{(b - a + 1) A_{29}}. \quad (5)$$

The estimate used by Eq. (3) is $\hat{\rho}$ evaluated over the order- m cell containing the region’s centre, with m chosen so that $s_m \approx 4r$: large enough to contain enough rows to be a density, small enough to be local.

Three things are worth stating plainly. The quantity is a density per unit solid angle, not a selectivity: the planner’s own selectivity estimate for the rewritten range conditions is computed separately by PostgreSQL from the same histogram. The interpolation in Eq. (4) assumes rows are spread uniformly inside a bucket, which is exactly what fails for a cluster smaller than a bucket – hence the area guard of Sect. 2.2. And the estimate can only be as local as a bucket: with $B = 1000$ and $N = 10^7$ a bucket holds 10^4 rows, so below that scale Eq. (5) returns the average over a region containing 10^4 rows, which is the correct answer to a different question. Section 6.11 measures how close $\hat{\rho}$ lands to the truth, and at which scales the truth itself is defined.

2.4 Classifying cells: what is bounded and what is not

A cell must be classified against the region as inside, outside or straddling its boundary. The three outcomes are not symmetric. A cell called *outside* is dropped from the covering, so if that is ever wrong

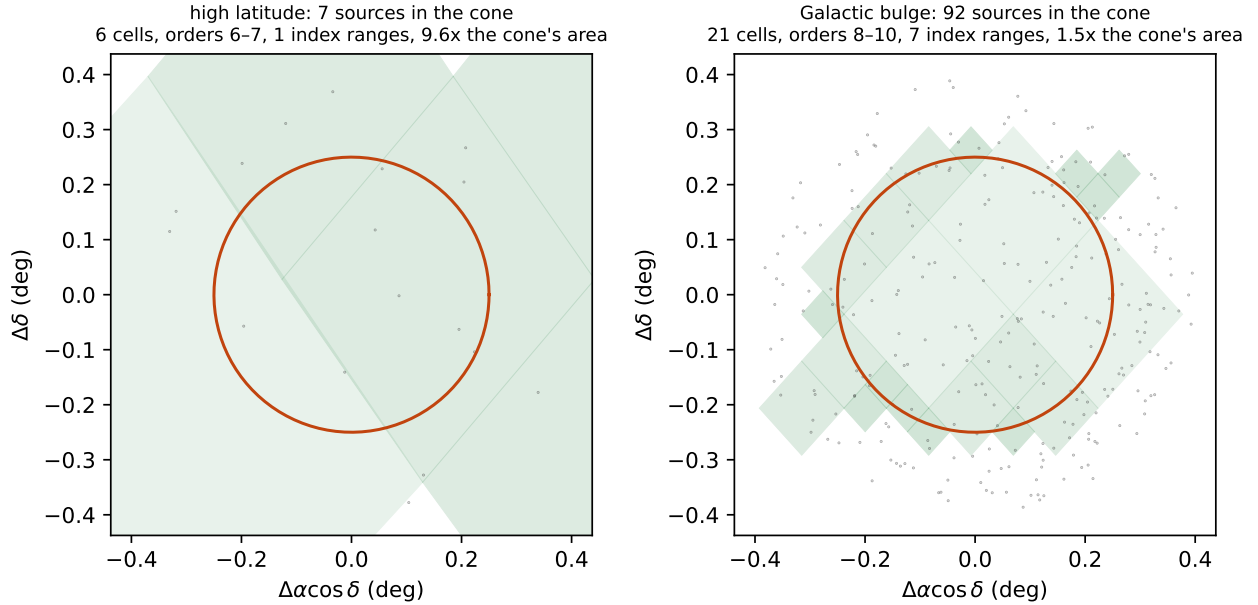


Figure 1: The same 0.25° cone covered in two places on the same catalogue. *Left*: at high Galactic latitude the cost model buys one index range of large cells, covering 5.6 times the cone’s area, because the 11 sources in the cone leave almost nothing to exclude. *Right*: in the Galactic bulge the same cone holds 1183 sources, and the model cuts two orders deeper – 23 cells in 8 ranges, 1.3 times the cone’s area – because there the false positives cost more than the ranges save. Grey dots are catalogue sources, the heavy circle the query cone, and the shaded quadrilaterals the covering cells, drawn from their own corners.

the query silently loses rows. A cell called *inside* is taken whole, and a cell called *straddling* is refined or taken whole; both produce a superset of the region, and the exact predicate that the rewrite conjoins to the index conditions (Sect. 3) removes whatever the covering over-included. Only the outside decision therefore needs a bound rather than an estimate.

The bound used is

$$R_{\text{cell}}(k, p) = \min(\text{maxpixmap}(k), \max_i \angle(\mathbf{c}_{k,p}, \mathbf{v}_i)), \quad (6)$$

where $\mathbf{c}_{k,p}$ is the cell’s centre and $\mathbf{v}_i$ its four corners. The first term is the analytic worst case over the sphere used by the reference HEALPix implementation; the second is the cell’s own extent, which is smaller for most cells. A cell is outside when $\angle(\mathbf{q}, \mathbf{c}_{k,p}) - R_{\text{cell}} > r + \epsilon$ for a cone of centre $\mathbf{q}$ and radius r , and correspondingly for each of a polygon’s bounding half-spaces. The margin $\epsilon = 2 \times 10^{-13}$ rad (40 pas) covers the accumulated floating-point error of the handful of dot products, cross products and inverse trigonometric calls involved, each carrying a few units in the last place; it is five orders of magnitude below the 0.4 mas resolution of an order-29 cell, so it costs nothing.

The inside test may be, and is, tighter: it uses the distances from the query centre to the cell’s own corners and edge midpoints directly.

We draw attention to a false start, because it is the kind of error brute-force testing exists to catch. An earlier version of this work also decided *outside*

from the distances to the chords between corners, inflating them by four times the departure of the edge from its chord measured at the edge midpoint. That is not a bound. Where an edge crosses its chord plane near the midpoint the measured departure collapses towards zero while the true departure does not, and a dense scan of every order finds the ratio between them reaching 64 (at order 18). The resulting error was about 10^{-8} rad, far below a leaf cell and invisible to every test we had, but it was an assumption sitting where the paper claims a guarantee. Replacing it with Eq. (6) costs 2.5% more index ranges per covering (8.1 to 8.3 on the validation suite) and no measurable time.

2.5 Regions as stored objects

A footprint is covered by a small number of cells written in the NUNIQ encoding of the MOC standard and stored one per row in an ordinary B-tree. The footprints containing a given position are those whose cell is one of that position’s ancestors, a short list of equality lookups; the exact geometry is then applied to the candidates. Since a MOC’s cells are disjoint by construction, no duplicate elimination is needed. Refinement of these coverings stops when cells become negligible next to the region, which keeps the ancestor list short.

2.6 The query surface

The extension exposes ADQL’s own vocabulary: types `skypos` and `skyregion` and functions `point`, `circle`, `box`, `polygon`, `contains`, `intersects`, `distance`, `area`, `coord1`, `coord2` and `centroid`, so that a translator emits the standard’s spelling rather than a dialect. The operator form `pos <@ circle(...)` is the one a planner can answer from an index; the function form returns the same rows.

Beside the geometry, archives need the astrometry that decides *where* a source is at the epoch being asked about. `skycell` implements the IVOA UDF registry’s names – `ivo_epoch_prop`, `ivo_epoch_prop_pos`, `ivo_apply_pm`, `ivo_healpix_index`, `ivo_healpix_center` – with the rigorous epoch propagation of the Hipparcos catalogue (ESA 1997), in which the star follows a straight line in space so that parallax, proper motion and radial velocity all evolve together. Frame conversions between ICRS, Galactic and ecliptic coordinates are provided for the same reason.

An index built on catalogue positions cannot know where a star has moved to, so a proper-motion-aware cone search is written as an indexable cone widened by the largest motion of interest, intersected with an exact test on the propagated position.

Table 2 lists the surface and, since the evidence for each part of it differs, what supports it.

3 Implementation

The extension is written in C against the PostgreSQL 15+ server API and consists of a geometry layer with no database dependencies – HEALPix arithmetic, region predicates and the covering – and a layer that connects it to the planner.

The connection is a *planner support function* (`SupportRequestSimplify`), the mechanism PostgreSQL uses for `STDWithin`. Before path generation, a predicate such as `point('ICRS', ra, dec) <@ circle('ICRS',  $\alpha$ ,  $\delta$ ,  $r$ )` is replaced by a disjunction of range conditions on the key expression, conjoined with the exact test:

```
(cell BETWEEN lo1 AND hi1
OR cell BETWEEN lo2 AND hi2 OR ...)
AND skycell_in_region(point(ra, dec),
                      circle(...), 0.29)
```

The key expression is not written by the user: the support function builds the candidate expression from the predicate’s position argument and keeps it only if the relation actually carries an index on it, so the same query text works whether positions are stored as two columns or as a `skypos` column. Because the index conditions are then ordinary range predicates, the planner estimates their selectivity from the same histogram, and the last argument of the exact test carries the fraction of the covering that

the region occupies, so that the product of the two estimates is close to the truth.

Two caches matter for planning cost. The density model and the identity of the matching index are cached per backend and dropped whenever statistics, the relation or the relation cache change; coverings are memoised by region and statistics source, since a service plans the same cone many times. Measured on a TAP corpus, the first cache is worth 45–70 μ s per plan and the second up to 50 μ s for larger regions.

For joins, where the region is not a constant, the rewrite emits a fixed number of range slots computed per outer row – the structure `q3c_join` uses – while a set-returning function `skycell_cone_ranges()` offers the alternative of a variable number of ranges through a `LATERAL` join. The two differ in plan shape rather than geometry, and Sect. 6.7 shows the difference is large.

Ingestion and maintenance. The index is an ordinary B-tree on an expression, so its maintenance behaviour is PostgreSQL’s: an insert adds one 8-byte key, updates that do not move a source do not touch it, and there is no rebalancing peculiar to the method. Two operational points differ from a geometry index. Bulk ingestion benefits from loading in key order, since that is also the heap order the queries exploit, and the cost of *not* doing so is the usual loss of locality rather than a degraded index. And the covering depends on the histogram, so statistics matter more than for a GiST index: a relation whose density structure has changed substantially since the last `ANALYZE` will be covered for the sky it used to have. The guard of Sect. 2.2 bounds the damage, and `ANALYZE` on the 10-million-row corpora here takes 0.5–1.4 s.

4 What is proven, and what is tested

4.1 A conservative-covering statement

Proposition. Let R be a cone or convex polygon and let $\mathcal{C}$ be the covering computed for it. If (i) every point of a HEALPix cell at order k lies within $\text{maxpixrad}(k)$ of that cell’s centre, and (ii) every angle computed by the classification carries an absolute error below ϵ , then every order-29 cell that intersects R lies in one of the ranges of $\mathcal{C}$.

Proof sketch. The covering is produced by a descent that starts from cells whose union is the whole sphere and, at each step, either keeps a cell, drops it, or replaces it by its four children, whose union is the parent. The union of the kept cells is therefore reduced only where a cell is dropped, which

Table 2: The query surface, and the evidence for each part. ADQL spellings are used unchanged; `skycell_`-prefixed aliases exist for schemas where a name collides with one in `pg_catalog`. *Benchmarked*: measured in Sect. 6. *Verified*: every indexed answer required to equal a sequential scan with the exact predicate, over the query sets of Sect. 4. *Exercised*: correctness checked by round trip or against published values, performance not characterised.

Signature	Meaning	Evidence
<i>Geometry (ADQL)</i>		
<code>point(frame, ra, dec)</code>	position $\rightarrow$ <code>skypos</code>	verified
<code>circle(frame, ra, dec, r)</code>	cone $\rightarrow$ <code>skyregion</code>	benchmark
<code>box(frame, ra, dec, w, h)</code>	box $\rightarrow$ <code>skyregion</code>	verified
<code>polygon(frame, ra, dec, ...)</code>	convex polygon	benchmark
<code>contains(a, b), intersects(a, b)</code>	1 or 0	verified
<code>distance(a, b), area(r)</code>	degrees, square degrees	exercised
<code>coord1/coord2/centroid</code>	components, centre	exercised
<code>pos <@ region</code>	indexable containment	benchmark
<i>Astrometry (IVOA UDF registry)</i>		
<code>ivo_epoch.prop(...)</code>	six parameters propagated	exercised
<code>ivo_epoch.prop.pos(...)</code>	position only	exercised
<code>ivo_apply.pm(...)</code>	proper motion only	exercised
<code>ivo_healpix_index/center</code>	cell of position, and back	verified
<code>icrs2gal, gal2icrs</code>	frame conversion	exercised
<code>icrs2ecl, ecl2icrs</code>	frame conversion	exercised
<i>Index and covering</i>		
<code>skycell_ang2cell(ra, dec)</code>	the order-29 key	benchmark
<code>skycell_cone/poly(cell, ...)</code>	indexable predicates	benchmark
<code>skycell_cone_ranges(...)</code>	ranges as rows (<code>LATERAL</code>)	benchmark
<code>skycell_cone_moc(...)</code>	covering as NUNIQ cells	benchmark
<code>skycell_ancestors(cell, ...)</code>	a position's containing cells	benchmark
<code>skycell_cover_info(...)</code>	covering diagnostics	exercised

happens only when it is classified outside. Taking $R_{\text{cell}} = \text{maxpixrad}(k)$ in Eq. (6), every point $\mathbf{x}$ of cell (k, p) satisfies $\angle(\mathbf{x}, \mathbf{c}_{k,p}) \leq R_{\text{cell}}$ by (i); hence if $\angle(\mathbf{q}, \mathbf{c}_{k,p}) - R_{\text{cell}} > r + \epsilon$, then by the triangle inequality on the sphere and (ii) every point of the cell is farther than r from $\mathbf{q}$, so the cell does not intersect the cone. The polygon case is the same argument applied to each bounding half-space. Truncation by the range cap, the area guard or the step budget only stops refinement, which keeps a cell whole, so it cannot remove a cell either. A cell containing a point of R intersects R and is therefore never dropped. $\square$

The proposition is conditional, and both conditions deserve their status. Condition (i) is a property of HEALPix, not of this work: the reference implementation's `maxpixrad` is analytic, and we verify it numerically over 1.6×10^6 sampled boundary points at every order, the largest observed ratio of centre-to-point distance to `maxpixrad` being 0.9999. The tighter corner term in Eq. (6) additionally assumes that the extremum of the distance from the centre over a cell is attained at a corner, which the same scan supports (largest observed ratio 0.96) but which we have not proven; a deployment that wants the proposition to rest on (i) alone can set `skycell.exact_cells` to off, at the cost quantified in Sect. 2.4. Condition (ii) is an assumption about floating-point arithmetic rather than a theorem: ϵ is chosen five orders of magnitude above the plausible

accumulated error and five below the leaf cell.

4.2 Brute force

Because a covering that misses a cell silently loses rows, the implementation is checked by exhaustion at three levels.

The HEALPix layer is checked against known cell centres, round trips between positions and cells at all 30 orders, the nesting property of Eq. (1), equal area ($\chi^2/\nu = 1.14$ over 768 base cells), and the bounds of Sect. 2.4, over 1.6×10^6 sampled points including the poles, the boundary between the polar and equatorial zones, and the base-pixel edges.

The covering is checked against a 2-million-point clustered catalogue for cones from $1''$ to 120° and for convex polygons; against 30 000 adversarial cones – at the poles, across the zone boundary, at $\alpha = 0$ and on face edges, with radii from 0.02 mas to 10° – with 200 points sampled inside each, half of them within a part per billion of the rim; and against 7266 adversarial polygons at the same awkward places, with 866 537 points drawn inside them, again half of them a hair inside an edge or a vertex. All assert zero false negatives, in both the seeded and the unseeded covering path. This is the test that caught the error described in Sect. 2.4.

At the SQL level, two regression suites require that every indexed answer equals the answer from a

sequential scan with the exact predicate: 240 cones in two configurations, 60 polygons, both join forms and the MOC footprint lookup in the first; the ADQL surface, the operator and function spellings over 120 random regions, the presence of index plans (and their absence when the region is not constant), epoch-propagation round trips and frame conversions in the second.

5 Benchmark

5.1 Two corpora

The first corpus is 10 million synthetic sources with a density contrast resembling a real optical survey: 35% uniform on the sphere, 45% in a Galactic disk (longitude uniform, latitude exponential with a 3° scale), 12% in a bulge (Gaussian, 7°), and 8% in 200 clusters with dispersions from 0.02° to 0.5° . Positions are drawn deterministically from a seed. We call it the *designed* corpus, because its density structure was chosen by the authors of the method under test, which is exactly the objection such a corpus invites.

The second corpus removes that objection as far as we can. We queried the ESA Gaia archive for the number of Gaia DR3 sources in every HEALPix cell at order 9 (3 145 728 cells of 0.11° ; the counts sum to 1 811 709 771, the published DR3 total), and drew 10 million positions from that density field: each cell receives a number of sources proportional to its Gaia count, placed uniformly inside it by choosing a random order-29 descendant, which is exact because the descendants are equal area. The result has the real sky’s crowding – the disk, the bulge, the Magellanic Clouds, the scanning-law texture – down to the 0.11° scale of the map, below which it is locally Poisson. It is not a Gaia catalogue: no real positions are used, and structure finer than a map cell is absent by construction. What it tests is whether the cost model, which reads density at scales of a few times the query radius, behaves the same on a density field the authors did not invent. We call it the *Gaia* corpus.

5.2 Targets and layout

Q3C 2.0.5, pgSphere 1.5.2 and `skycell` each hold the same logical rows in their own natural layout, all three physically ordered along a space-filling curve so that heap locality is held equal (Table 3). Queries are the documented interface of each extension: `q3c_radial_query`, `spoint <@ scircle` and `skycell_cone`. Row counts are compared on every query.

5.3 Protocol

The measurement is organised as *trials*. A trial is one query centre and radius; within a trial all three meth-

ods answer the same query back to back, in an order drawn at random for that trial, and the position each method drew is recorded so that an order effect can be measured rather than assumed away. Trials are repeated five times over the whole query set, and the query set is traversed in a random order each repetition. The analysis is paired: for each query we take the median over repetitions per method and form the ratio between methods for that same query, so that the query’s own difficulty cancels; the reported interval is a percentile bootstrap (4000 resamples) over queries. A ratio is called a difference only when its interval excludes 1.

Everything is measured in two cache regimes. *Warm*: an untimed pass over the whole query set precedes the measurement, so no method is measured on a colder cache than another. *Cold*: the server is restarted, so its 2 GB of shared buffers is empty, and the virtual machine’s page cache is dropped, so the data must come from storage; only the first repetition is used, since a second would no longer be cold.

Separately, a randomized pass under `EXPLAIN (ANALYZE, BUFFERS)` records planning time, execution time, buffer pages and the planner’s estimate against the actual count, so that planning and execution can be reported apart from each other.

Cone radii span $1''$ to 3° with 16–400 queries each, half of the centres drawn from catalogue positions (hence density-weighted) and half uniform on the sky. Cross-matches join a 200 000-row probe table – half catalogue sources displaced by $\sim 0.3''$, half uniform – to the catalogue. Footprint queries ask which of 20 000 stored circles contain each of the 200 000 probes.

5.4 Reproducibility

Table 3 records the configuration in the detail needed to repeat or contest the measurement. Everything runs on one machine, with one core doing one query.

5.5 What this protocol still does not control

The corpora are synthetic, one designed by the authors and the other derived from a real density map but not from real positions. One machine, one storage device, one PostgreSQL configuration. Absolute times are properties of that machine; only the comparisons are meant to travel. The cold regime drops the page cache but cannot empty the device’s own cache. And the three extensions are compared as configured here: a different Q3C query formulation or pgSphere index configuration could move their numbers, and Sect. 6.1 reports one such effect that dominates Q3C’s small-cone results.

Table 3: The measured configuration.

Item	Value
Host	Apple silicon laptop, 11 cores, Docker Desktop Linux VM, 8 GB
Storage	NVMe SSD, through the VM’s file system
PostgreSQL	18.6; <code>shared_buffers</code> 2 GB, JIT off, <code>max_parallel_workers_per_gather</code> 0
Extensions	Q3C 2.0.5, pgSphere 1.5.2, skycell 0.4
<code>range_cost</code>	derived per relation (Eq. 7); 116 for the catalogues here
Rows	10^7 per corpus (Gaia corpus: 9999729); the Gaia corpus also at 5×10^7 (50 027 391)
Q3C	<code>cat_q3c(id, ra, dec, mag)</code> , B-tree on <code>q3c_ang2ipix(ra, dec)</code> , heap ordered by the same
pgSphere	<code>cat_sphere(id, ra, dec, mag, pospoint)</code> , GiST on <code>pos</code> , heap ordered by the HEALPix key
skycell	<code>cat_cell(id, ra, dec, mag, cellint8)</code> , B-tree on <code>cell</code> , statistics target 1000, heap ordered by <code>cell</code>
Maintenance	<code>VACUUM (FREEZE)</code> and <code>ANALYZE</code> on all three before measuring
Repetitions	5 (warm), 1 (cold); 1456 queries per repetition
Uncertainty	percentile bootstrap, 4000 resamples, over queries

6 Results

All three methods return identical row counts on every query of the grid. In the cross-matches pgSphere returns one or two more pairs out of $\sim 100\,000$, consistent with its 10^{-9} rad comparison tolerance counting sources exactly on the rim.

6.1 Cone searches

Table 4 and Fig. 2a are the primary result: paired ratios with intervals, on both corpora, warm and cold.

Warm, `skycell` is faster than `pgSphere` by 6–11% for radii from $1''$ to $1'$ and by 31–37% at 3° , on both corpora. Between $6'$ and 1° the two are, on the designed corpus, indistinguishable; on the Gaia corpus `skycell` is 3% *slower* at $6'$ (interval [1.00, 1.08]) and 8–15% faster at $30'$ and 1° . This is a smaller and more ragged advantage than a phase-ordered measurement suggests (Sect. 6.4), and it is the number we stand behind.

Cold, the picture is different and more favourable: `skycell` is faster at every radius on both corpora, by 5% at $1'$ and by a factor 2–2.5 at $30'$ and beyond. The reason is visible in the buffer counts of Table 5: at 1° `skycell` touches 55 pages against `pgSphere`’s 154. When those pages are in memory the difference is a few microseconds of memory traffic; when they must be read it is the whole cost of the query. An index that wastes less area is worth most exactly when the

data does not fit in cache, which is the regime a large archive operates in.

Against Q3C, `skycell` is faster by a factor 4–50 depending on radius and cache state. The gap at small radii has a specific and reportable cause: Q3C 2.0.5 expands every `q3c_radial_query` into 100 key ranges regardless of radius, which costs about 300 buffer pages and 1.2 ms of planning before any row is read (Table 5). This is a property of that version’s query path rather than of the cube quad-tree; Q3C’s cross-match path, which uses four ranges per probe, does not pay it, and Sect. 6.7 shows it performing accordingly.

The order effect the randomization exists to control is small but real: averaged over all radii on the designed corpus, going first rather than last costs `pgSphere` 19% and `skycell` –5% warm, while Q3C is flat. Since methods are assigned to positions at random, this inflates the variance of the ratios rather than biasing them.

6.2 Does the advantage grow with the catalogue?

Ten million rows is 0.55% of Gaia DR3, and the argument that an index matters most when it does not fit in memory is an argument about scale. We therefore repeated the whole protocol on a 50-million-row Gaia-density corpus, built identically. The point of that size is specific: `pgSphere`’s GiST index is then 3442 MB against a 2 GB `shared_buffers`, so it no longer fits, while `skycell`’s B-tree is 1072 MB and still does.

The answer (Table 7) is not uniform, and it is more interesting than the prediction. For regions of $30'$ and larger the advantage grows markedly with the catalogue: at 1° the warm ratio improves from 0.85 to 0.64, and the cold ratio from 0.53 to 0.27 – a factor 3.7. For cones of an arcsecond to a few arcminutes it moves the other way, and warm, at 50 million rows, `skycell` is *slower* than `pgSphere` at $1'$ (1.07) and $6'$ (1.13).

The planning column of Table 7 explains both halves. `skycell`’s planning time grows with source density – 0.014 to 0.020 ms at $1'$, 0.019 to 0.031 ms at $6'$ – because a denser sky makes Eq. (3) cut finer and emit more range arms, each of which is a separate index path for the planner to cost. `pgSphere`’s planning is flat at 0.009 ms whatever the density. At $6'$ on 50 million rows `skycell` still executes faster (0.042 against 0.049 ms) and loses anyway, because it gave up 0.022 ms before the first row was read. At 1° the execution gap (0.74 against 1.26 ms, 153 against 464 buffer pages) is an order of magnitude larger than the planning penalty, and grows with the catalogue, so the penalty stops mattering.

This is the sharpest statement we can make about where the method belongs: its cost is paid per query and its benefit is paid per row scanned, so it wins

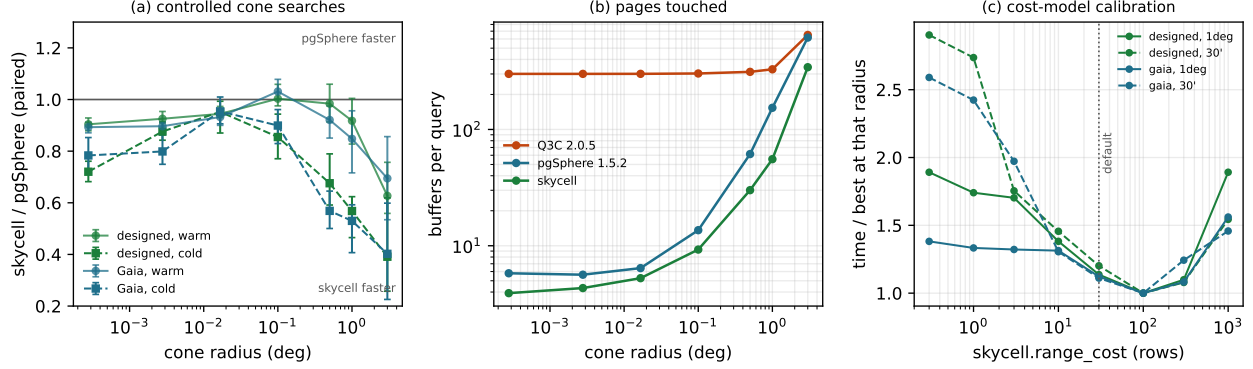


Figure 2: (a) The controlled comparison: the paired ratio of **skycell** to pgSphere against cone radius, with 95% bootstrap intervals, for both corpora and both cache regimes. Below the line skycell is faster. The warm advantage disappears around 0.1° and returns at degree scales; with caches dropped it holds at every radius. (b) Buffer pages touched per query, which is what the cold curves are made of: at 1° skycell reads a third of pgSphere’s pages, and Q3C pays about 300 pages at any radius because version 2.0.5 always expands a radial query into 100 key ranges. (c) The cost model’s one free parameter, swept over four decades and normalised to the best setting at each radius: the optimum is near 100 on both corpora, the default of 30 costs 11–21%, and the curve is flat-bottomed over a decade either side.

Table 4: Cone searches: paired ratio of **skycell** to each rival, with a 95% percentile-bootstrap interval over queries. A value below 1 means **skycell** is faster. **Bold** marks an interval that excludes 1 with skycell faster; *italic* marks one that excludes 1 with skycell slower; plain type means the two are not distinguishable. *n* is the number of distinct queries at that radius; each is answered five times per method (warm) or once (cold), all three methods in a randomized order within each trial.

		designed corpus		Gaia corpus	
radius	n	warm	cold	warm	cold
<i>skycell / pgSphere</i>					
1''	400	0.90 [0.88, 0.93]	0.72 [0.68, 0.76]	0.89 [0.87, 0.92]	0.78 [0.72, 0.85]
10''	400	0.93 [0.91, 0.96]	0.88 [0.84, 0.92]	0.90 [0.88, 0.91]	0.80 [0.75, 0.86]
1'	300	0.94 [0.93, 0.97]	0.95 [0.87, 1.00]	0.93 [0.90, 0.96]	0.95 [0.90, 1.01]
6'	200	1.00 [0.98, 1.06]	0.85 [0.77, 0.94]	<i>1.03</i> [1.00, 1.08]	0.90 [0.83, 0.96]
30'	100	0.98 [0.91, 1.06]	0.68 [0.59, 0.79]	0.92 [0.85, 0.98]	0.57 [0.50, 0.65]
1°	40	0.92 [0.83, 1.00]	0.57 [0.47, 0.62]	0.85 [0.72, 0.96]	0.53 [0.41, 0.59]
3°	16	0.63 [0.56, 0.76]	0.39 [0.26, 0.62]	0.69 [0.53, 0.86]	0.40 [0.22, 0.58]
<i>skycell / Q3C</i>					
1''	400	0.02	0.14	0.02	0.14
1'	300	0.02	0.18	0.02	0.19
30'	100	0.06	0.42	0.06	0.41
1°	40	0.09	0.59	0.13	0.51
3°	16	0.27	0.75	0.37	0.66

Table 5: Planning and execution separated, with buffer pages and the error of the planner’s row estimate (median of $\max(\text{est}/\text{act}, \text{act}/\text{est})$), from a randomized EXPLAIN (ANALYZE, BUFFERS) pass on the designed corpus.

radius	method	plan (ms)	exec (ms)	buffers	est. err
1''	Q3C	1.230	0.213	300	1.00
	pgSphere	0.008	0.017	5	1.00
	skycell	0.015	0.007	4	1.00
30'	Q3C	1.242	0.291	313	1.92
	pgSphere	0.006	0.106	61	2.92
	skycell	0.034	0.075	30	1.16
1°	Q3C	1.224	0.402	329	1.89
	pgSphere	0.006	0.324	154	2.78
	skycell	0.062	0.145	55	1.22

exactly where enough rows are scanned to amortise a covering, and that boundary moves *up* in radius as the catalogue grows denser, not down. On a Gaia-sized catalogue, a small cone search is not the workload this index improves; a degree-scale selection, a polygon or a cross-match is.

6.3 Does the advantage grow with scale?

Every number above is from a 10-million-row table, which is 0.55% of Gaia DR3. Since the index sizes differ by a factor three, the obvious extrapolation is that at larger scales pgSphere’s index stops fitting in memory while skycell’s still does, and that the advantage should therefore widen. Extrapolating two orders of magnitude from a single scale is not evidence, so we measured a second scale: the same Gaia density field resampled to 50 million rows, the same protocol, the same machine. At that size pgSphere’s GiST index is 3442 MB and skycell’s B-tree 1072 MB, against `shared.buffers` of 2048 MB – so one index fits and the other does not.

Table 7 gives both scales side by side, and the answer divides sharply by radius.

For regions of 30' and larger the advantage grows, and grows more cold than warm: at 1° the ratio moves from 0.85 to 0.60 warm and from 0.53 to 0.31 cold, and at 3° from 0.40 to 0.23 cold. The buffer counts show why: at 1° and 50 million rows pgSphere touches 1848 pages against skycell’s 387, a ratio of 0.21 where at 10 million rows it was 0.36. The page advantage itself widens with scale, and once the pages have to be read rather than hit, that is the whole query.

For cones of 1' and smaller nothing improves, and 1' gets slightly worse (0.93 to 1.02, no longer distinguishable from pgSphere). The reason is visible in the same table: at those radii both methods touch five to eight pages whatever the table size, so there is no scan to shorten, and what remains is skycell’s 0.021–0.023 ms of planning against pgSphere’s

0.008 ms. Five times the density also means a finer covering – $s_* \propto \rho^{-1/2}$ from Eq. (3) – hence slightly more ranges to plan and, at 1', slightly more pages touched than pgSphere (a ratio of 1.20).

The honest summary for a large catalogue is therefore not “skycell is faster” but: *the larger the table and the larger the region, the more it wins; for arcminute-scale cone searches it is a wash at any size we measured, and scale does not change that.*

6.4 How much the protocol matters

Running the same queries one method at a time – each method’s whole grid, then the next – gives skycell/pgSphere ratios of 0.69, 0.77, 0.75, 0.87, 0.79, 0.44 and 0.57 from 1'' to 3° on the designed corpus: an apparently uniform advantage of 13–56%, where the controlled protocol finds 6–11% at small radii and no difference at all between 6' and 1°. The phase-ordered arrangement flatters whichever method is measured later, and that was skycell. We report this because the difference between the two arrangements is larger than most of the effects being measured, and because an earlier version of this work presented the phase-ordered numbers as its primary result.

6.5 Index size and build time

The B-tree of 64-bit keys is 214 MB and builds in 1.4–2.3 s; pgSphere’s GiST is 685 MB and builds in 42–44 s (Table 8). For an archive this is an operational number: it is what an ingest, a restore or a schema migration has to pay, and it is the one result here that is not sensitive to protocol.

6.6 What the index costs at Gaia’s scale

Index size is the one result in this paper that is insensitive to protocol, and it is also the one that decides deployments, so it is worth carrying to the scale an archive actually runs at. The cost per row is flat: between 10^7 and 5×10^7 rows it changes by 0.5% for pgSphere and not at all for the two B-trees (Table 9), which is what the structures predict – an 8-byte key with its tuple header in a B-tree, a bounding volume per entry in a GiST tree.

Extrapolating to the 1.81×10^9 sources of Gaia DR3 gives the right-hand columns of Table 9. The comparison that matters there is not one index against another but one *deployment* against another. pgSphere answers cone searches and stores regions; `q3c.join` is what PostgreSQL archives use for cross-matching, and Sect. 6.8 measures it 2.4 times faster than pgSphere at that. An archive that wants both therefore carries both indexes today. skycell answers both from one, and that one is the smaller kind.

Table 6: The same protocol at two catalogue sizes on the Gaia density field: paired ratio of **skycell** to pgSphere with 95% bootstrap intervals, and the planning/execution/buffer medians behind them. At 5×10^7 rows pgSphere’s index (3442 MB) exceeds **shared_buffers** while **skycell**’s (1072 MB) does not. Bold: interval excludes 1, **skycell** faster; italic: interval excludes 1, **skycell** slower.

radius	warm		cold		plan (ms)		exec (ms)		buffers	
	10^7	5×10^7	10^7	5×10^7	pgS	sky	pgS	sky	pgS	sky
1''	0.89	0.94	0.78	0.69	0.009	0.016	0.019	0.009	5	4
10''	0.90	1.00	0.80	0.90	0.009	0.016	0.019	0.011	5	5
1'	0.93	<i>1.07</i>	0.95	0.92	0.009	0.020	0.023	0.017	6	6
6'	1.03	<i>1.13</i>	0.90	0.81	0.009	0.031	0.049	0.042	14	19
30'	0.92	0.78	0.57	0.49	0.009	0.062	0.439	0.246	106	66
1°	0.85	0.64	0.53	0.27	0.009	0.081	1.260	0.744	464	153
3°	0.69	0.59	0.40	0.24	0.011	0.130	9.604	5.550	1078	634

The planning, execution and buffer columns are medians at 5×10^7 rows, from the randomized EXPLAIN pass.

Table 7: The same Gaia density field at two scales: paired ratio of **skycell** to pgSphere, with 95% bootstrap intervals. **Bold**: interval excludes 1, **skycell** faster. At 50 million rows pgSphere’s index (3442 MB) exceeds **shared_buffers** (2048 MB) while **skycell**’s (1072 MB) does not.

radius	warm		cold	
	10^7 rows	5×10^7	10^7 rows	5×10^7
1''	0.89	0.89	0.78	0.71
10''	0.90	0.94	0.80	0.88
1'	0.93	1.02	0.95	0.95
6'	<i>1.03</i>	1.00	0.90	0.89
30'	0.92	0.72	0.57	0.49
1°	0.85	0.60	0.53	0.31
3°	0.69	0.58	0.40	0.23
<i>buffer pages, 5×10^7 rows (pgSphere / skycell)</i>				
1': 7 / 8 1°: 1848 / 387				

Table 8: Storage and build cost for the same 10 million rows (Gaia corpus; the designed corpus agrees to within 5% except for **ANALYZE**). Table sizes differ because each method stores its own position representation alongside the catalogue columns.

method	table (MB)	index (MB)	build (s)	ANALYZE (s)
Q3C	575	214	2.2	0.23
pgSphere	731	685	42.3	0.46
skycell	652	214	1.4	0.57

Table 9: Index cost per row, measured at two scales, and what it implies for a catalogue the size of Gaia DR3 (1.81×10^9 sources). Build times scale linearly from the 5×10^7 measurement, which is optimistic for all three equally. The last column is the smallest conventional server memory that holds the index resident with room left for the heap’s hot pages.

index	B/row		at 1.81×10^9		
	10^7	5×10^7	GiB	build	GiB
skycell	22.5	22.5	38	14 min	64
Q3C	22.5	22.5	38	13 min	64
pgSphere	71.8	72.2	122	2.7 h	192
pgSphere + Q3C	—	—	160	2.9 h	256

The memory column is the operational point. A positional index is only as fast as its residency: the measurements of Sect. 6.1 differ by a factor two to three between the warm and the cold regime, and which regime a server is in is decided by whether the index fits beside the heap’s working set. At DR3 scale a 64 GiB server holds **skycell**’s 38 GiB index with room to spare; pgSphere’s 122 GiB does not fit in 128 GiB once the heap is accounted for, and the pair needed to cover both cone searches and cross-matching wants 256 GiB. That is a factor four in the memory the machine must have, which at these sizes is a procurement decision rather than a tuning one – and it is exactly the regime where our cold-cache measurements put **skycell** furthest ahead (0.31 to 0.53 of pgSphere’s time at 30' and beyond).

Four caveats. The extrapolation is linear and validated only over a factor five in row count, though the per-row cost is constant to 0.5% across it. These are positional indexes alone: a real archive’s table, its other indexes and its ancillary products dwarf them. A partitioned deployment changes the arithmetic, since only the partitions in play need be resident. And the two B-trees are the same size, so the comparison is really one geometry index plus one integer index against one integer index – the saving comes from not needing the geometry index at all, which rests on **skycell** answering the region queries pgSphere is kept for, as Sects. 6.1 and 6.7 measure.

6.7 Polygons, cross-matches and footprints

Convex polygons of 0.05° – 2° take a median 0.281 ms on the Gaia corpus against pgSphere’s 0.767 ms and Q3C’s 1.864 ms, and 0.142 ms against 0.631 and 1.526 on the designed corpus: a factor 2.7–4.4 either way; pgSphere’s mean (4.2 ms) and 95th percentile (21 ms) are far above its median, which the per-query records trace to polygons whose bounding volume is a poor fit.

6.8 Cross-matching

Cross-matching a user’s uploaded target list against the catalogue is the operation the Gaia archive lists among its server-side capabilities (Salgado et al. 2017), and it is Q3C’s own speciality: `q3c_join` is what most PostgreSQL archives use for it. It therefore deserves the same protocol as the cone searches rather than a single end-to-end run, and getting that protocol right turned out to matter more here than anywhere else in this paper.

The probe table is split into eight blocks of 25 000 probes, and a block is the unit of pairing: within a block all four methods run back to back in an order drawn for that block. Cache residency has to be controlled explicitly. At 50 million rows the three catalogues and their indexes are about 14 GB against 2 GB of `shared_buffers`, so a block’s first touch is I/O bound – the same `skycell` block query takes 6.2 s cold and 0.32 s once its pages are resident. Whichever method runs first on a block therefore pays for the others, and that effect is larger than the difference between the methods: in a first version of this measurement, running first cost `skycell` a factor ten while leaving Q3C and pgSphere, whose own cost is several seconds, visibly unchanged. Each method is consequently warmed on its own query for its own block immediately before being timed, so that all four are measured in the same state and what is compared is the work each does.

Table 10 gives the result at 50 million rows. All four methods return identical row counts in every block.

`q3c_join` is the fastest of the two established methods by a factor 2.4, which is what its reputation predicts. `skycell`’s `LATERAL` form is level with it – 0.78 of its time at 1’’ with an interval that reaches 1.00, and 0.92 at 10’’, not distinguishable – and 2.5 to 3.3 times faster than pgSphere. The join form, which emits a fixed number of range slots per outer row in the manner of `q3c_join`, is level with Q3C at 1’’ and 35% slower at 10’’, where a fixed slot count fits the covering less well.

The claim this supports is the narrow one: on the operation Q3C is built for, `skycell` is Q3C’s equal rather than its better, while being two and a half to three times faster than pgSphere; and it reaches that position with the same index that answers the region queries of Sect. 6.1, which is the point of the design. We note also what an uncontrolled measurement would have reported. Our own earlier run, one method after another with no per-method warming, gave 14 s for `skycell` against 28 s for Q3C and 41 s for pgSphere – a factor two over Q3C that is an artefact of which method paid for the first touch of each block.

For stored footprints, the MOC-in-a-B-tree arrangement answers the same 200 474 point-in-region pairs in 0.76 s against pgSphere’s 0.97 s, with an in-

Table 10: Cross-matching 200 000 probes against 50 million sources, steady state: total time and the paired ratio to each rival over 16 block-repetitions. **Bold**: interval excludes 1, the method in the row is faster; *italic*: interval excludes 1, slower. All methods return identical row counts.

method	total (s)		ratio at 1’’	
	1’’	10’’	vs Q3C	vs pgS.
<code>q3c_join</code>	1.23	1.32	—	0.42 [0.36, 0.49]
pgSphere	2.95	3.22	<i>2.39</i> [2.04, 2.82]	—
skycell, slots	1.50	1.85	1.07 [0.89, 1.48]	0.44 [0.38, 0.55]
skycell, lateral	1.08	1.43	0.78 [0.62, 1.00]	0.30 [0.26, 0.42]

dex that is larger (9.5 MB against 1.1 MB) and slower to build (0.9 s against 0.03 s), since a region becomes several rows rather than one.

These three workloads are measured one method after another rather than in randomized trials, because each is a single query of seconds rather than a population of sub-millisecond queries; the ordering effect that Sect. 6.4 quantifies for cone searches applies to them, and their ratios should be read with that in mind.

6.9 Estimates, plans and joins

A spatial predicate is rarely alone. We ran three shapes – a cone with a catalogue cut, the same joined to an observation table, and a three-way join – through each method, recording the estimate, the plan strategy and the time separately, because they are different claims.

The estimate advantage carries over: for queries returning rows, the median estimate error at 30’–1° is 1.15–1.88 for `skycell` against 1.63–2.33 for Q3C and 2.50–3.00 for pgSphere.

The plans, however, are the same. Across 135 queries at 1’, 30’ and 1°, all three methods chose the same strategy (nested loop over index scans) for every query; `skycell`’s better estimate did not change what the planner decided. Only at 3°, where a cone returns ~15 000 rows, did one query out of sixteen cross a threshold: `skycell`’s estimate led to a hash join where pgSphere’s led to a nested loop, and that query ran 1.29 times faster. Execution times in the join shapes are otherwise within noise of pgSphere at 1’–1°.

The honest summary is that better estimates are demonstrated, better plans are not, and better join times are not. The value of an accurate estimate is insurance against the plan the optimiser would otherwise choose in a query we did not write; we have not shown that insurance paying off on a workload we did write.

6.10 The cost model’s one parameter, and how to stop setting it

Equation (2) has one free parameter, c_{range} . We first measured it, then removed it.

Measuring it. Table ?? sweeps c_{range} over four decades at 10^7 rows on both corpora. The curve is flat-bottomed: a factor of ten moves the median time by less than 25% anywhere between 3 and 300, and at $1''$ four decades move it by 10%, because there the covering is a single cell whatever the model says. Repeating the sweep at 5×10^7 rows flattens it further – at 1° every setting from 3 to 300 lies within 3% (0.70–0.72 ms), and at $30'$ the minimum sits near 60 with 30 and 100 within 12% of it. Cold, the differences vanish into the noise of a single pass entirely.

Deriving it. Because c_{range} is a ratio of two costs PostgreSQL already models, it need not be a constant at all. One more index range is a B-tree descent, which `btcostestimate` charges as $\lceil \log_2 N \rceil \cdot 50 \cdot c_{\text{op}}$ for the comparisons plus $(H + 1) \cdot 50 \cdot c_{\text{op}}$ for the pages; one false-positive row costs its index entry, its heap tuple, the exact predicate, and the heap page amortised over the rows that share it:

$$c_{\text{range}} = \frac{(\lceil \log_2 N \rceil + H + 1) 50 c_{\text{op}}}{c_{\text{tuple}} + c_{\text{itup}} + k c_{\text{op}} + c_{\text{page}}/n_{\text{pp}}}, \quad (7)$$

with $n_{\text{pp}} = \text{reltuples}/\text{relpages}$ the rows per heap page.

Every input is already at hand: `relpages` sits beside `reltuples` in the `pg_class` tuple the density model loads, and the cost factors are the planner’s own GUCs. `skycell.range_cost` defaults to -1 , meaning evaluate Eq. (7).

The last term is what makes this per-relation rather than per-installation. On the 10-million and 50-million-row catalogues, at 120 rows per page, Eq. (7) gives 116; on an ObsCore-shaped relation, whose wider rows pack 54 to a page so that each false positive costs more page traffic, it gives 93, asking for a finer covering. With the stock `random_page_cost` of 4.0 instead of this host’s 1.1 it gives 67. All three lie inside the flat bottom measured above, and all are well above the fixed 30 we had shipped.

What it buys. Not speed. Measured in the way that can actually answer the question – both settings placed in the same randomized trials, paired per query, so that no comparison crosses a run – the derived value and the old fixed 30 are indistinguishable at every radius on the 50-million-row corpus: ratios 0.93 to 1.08, every interval containing 1 (Table 11). What the derivation buys is that the parameter stops being a number someone has to choose, adapts to row width and to the storage the operator

Table 11: The derived c_{range} (116) against the previously shipped constant (30), both in the same randomized trials at 5×10^7 rows, warm, paired per query. No interval excludes 1.

radius	derived / fixed	der. vs pgS.	fix. vs pgS.
$1''$	1.02 [0.97, 1.06]	0.72	0.69
$10''$	1.00 [0.96, 1.03]	0.74	0.76
$1'$	0.97 [0.93, 1.06]	0.80	0.78
$6'$	1.07 [1.00, 1.19]	0.89	0.79
$30'$	0.95 [0.86, 1.05]	0.58	0.59
1°	1.08 [0.90, 1.23]	0.56	0.47
3°	0.93 [0.78, 1.03]	0.40	0.49

has described to the planner, and cannot silently be left at a value tuned for a different relation.

We report this at some length because our own first attempt to answer it was wrong. We compared a run at the derived value against an earlier run at 30 and read off a 15% penalty at 1° . Both runs were internally paired against pgSphere, but the comparison *between* them crossed an hour of machine drift – the same phase-ordering error this paper criticises in Sect. 6.4, committed on our own parameter. Inside a single set of trials the effect disappears.

The order it chooses, and how much that costs. The order the model picks can be compared with the best available order, because `skycell.force_order` makes every order measurable for the same query. Measured with the two orders interleaved per query, the model’s choice costs about 3% against one order finer at $6'$: it is close to optimal, and the flat cost curve of Table ?? is the reason.

We arrived at that number badly enough to be worth recording, because it bears on how the rest of this paper’s measurements should be read. Two earlier answers were wrong in opposite directions. The first, 0.8–2%, was measured through a defect in the covering cache, whose key did not include the forced order, so every order silently returned the same cached covering. With that fixed, a sweep across orders showed 30–48% – but the sweep visited the orders in ascending sequence, so each finer order ran on a cache the coarser ones had warmed, which is precisely the phase-ordering effect Sect. 6.4 quantifies for the method comparison. Interleaving the two orders per query removes it and leaves 3%.

Three implementation attempts sit behind that conclusion and are worth their cost. Scoring candidate orders on the covering they actually produce (`skycell.probe_orders`) required, in turn, putting the forced order into the cache key, gating the probe so it is not paid where a covering holds too few rows to repay it, and – the substantive one – merging each candidate’s gaps before counting its ranges, since a fine covering has many separate runs before merging and scoring it on that count charges for ranges the final plan never contains. With all three, the probe

Table 12: The histogram’s density estimate against the counted truth in the same footprint: median and 10th–90th percentiles of $\hat{\rho}/\rho_{\text{true}}$. Footprints holding fewer than 30 rows are excluded, which removes the 1’ row on the designed corpus entirely.

footprint (order)	rows in it	designed	Gaia
3 (1°)	22 548	1.02 [0.92, 1.16]	1.00 [0.84, 1.34]
4 (30’)	5452	1.02 [0.85, 1.54]	0.98 [0.78, 1.32]
7 (6’)	165	0.89 [0.44, 2.06]	0.91 [0.70, 1.30]
9 (1’)	39	—	0.94 [0.63, 1.04]
15 (1’')	0.5	no density to estimate	

does now choose finer coverings (area ratio 11.5 to 8.9 at 6’) and the query time does not change measurably, which is what a 3% headroom predicts. The machinery ships disabled.

6.11 The density estimate against the truth

The cost model’s input is $\hat{\rho}$ from Eq. (5). We compared it with the number of rows actually in the same footprint, counted (Table 12).

The estimator is unbiased where a density exists, and its scatter grows as the footprint empties – which is shot noise in the reference, not error in the estimate. Below about an arcminute the comparison stops meaning anything: the footprint the model reads holds half a row on average, so there is no local density to estimate. That is also the regime in which Sect. 6.10 shows the answer does not matter, because the covering is one cell whatever density is assumed. The equi-depth histogram is a poor two-dimensional density estimator in principle (Sect. 2.3); these measurements say that in practice it is good enough at the scales where its answer changes a decision.

7 Inside a TAP service, and where the crossover is

A benchmark of index operations is not a benchmark of an archive. We therefore ran the same build inside **egernia** (Delli Veneri & Salgado 2026), a TAP 1.1 server, against its published 500 096-row ObsCore corpus, in an A/B where one database held both indexes and two otherwise identical service processes differed only in how they translate CONTAINS: one to pgSphere, one to **skycell**. The service’s own harness interleaved the two targets rung by rung: six query classes (a mixed workload, a non-spatial control, two cone classes, a cone with time and calibration cuts, and an empty cone) at 1, 8 and 32 concurrent clients, three repetitions each, with a tie declared unless the 95% intervals are disjoint *and* the difference exceeds 10%.

Both targets pass the conformance gate (**taplint**, zero errors) and the cross-server agreement gate on

all eleven portable query classes, and the 203 cone queries of the corpus return identical rows through both translations.

The performance result is a tie: all 18 comparison cells fall inside the tie rule, with throughput ratios between 0.96 and 1.06. At the database level **skycell** is 1.2–1.45 times *slower* per query than pgSphere on that corpus.

Both facts have the same cause. A cone query over half a million rows returns tens to hundreds of rows and costs a fraction of a millisecond either way, against roughly 10 ms of CPU per request spent in the service’s own Python; the database is 1–3% of a request, so a difference of 0.08 ms in it cannot move the throughput of the service.

That explanation makes a testable prediction, and an earlier version of this work asserted it rather than testing it. We therefore built the same ObsCore-shaped relation – observations in groups of 128 products sharing a field centre, clustered by collection and field, which is what makes a GiST bitmap scan so effective on this data – at three sizes, and ran the same four cone classes against both indexes (Table 13).

The crossover is real, but it is not a single number, and one of its rows is not a number at all.

For the cone with catalogue cuts (Q07) and the degree-scale cone (Q06), **skycell** is ahead at every size and pulls further ahead as the relation grows: 0.84 to 0.71 for Q07 and 0.92 to 0.67 for Q06, reading a seventh of pgSphere’s buffer pages at Q06 and a quarter at Q07. The empty cone (Q12) is level. These are the classes where the scan is large enough for a better covering to matter, and they behave exactly as the catalogue results predict.

The small-cone class (Q05, 0.15°) is the one we cannot pin down, and we prefer to say so rather than quote the number the table happens to contain. Measured immediately after the relation is built, with both methods paired inside a trial, **skycell** is 2.16 times slower and the two intervals are disjoint ([0.269, 0.469] against [0.146, 0.242] ms). Measured on the same relation once it is thoroughly warm, with the same pairing and five repetitions, the same class comes out at 0.94 – level. Both are within-run paired measurements; they are measuring different cache regimes, and on this host, with 2 GB of shared buffers and several multi-gigabyte relations present, we were not able to make the small-cone ObsCore comparison reproducible to better than a factor of two. The decomposition is at least unambiguous: the deficit is in execution (0.324 ms against 0.161) and not in planning (0.034 against 0.009), and it is not the exact predicate – removing that predicate entirely changes the query time by about 1% on this relation, against 63% on the catalogue at a comparable radius.

An earlier version of this table was measured one method at a time rather than in randomized paired trials. It reported Q07 as a loss at every size (1.26,

Table 13: Crossover on an ObsCore-shaped relation, measured in randomized paired trials (30 centres, 3 repetitions, each method warmed on its own query first): total query time as the ratio `skycell/pgSphere`, with buffer pages at the largest size. Q05: 0.15° cone; Q06: 2° cone; Q07: 0.5° cone with calibration and time cuts; Q12: an empty $0.7''$ cone. The Q05 row is regime-dependent and should not be read as a single number (text).

class	skycell / pgSphere			buffers at 10^7	
	5×10^5	2×10^6	10^7	pgSphere	skycell
Q05	1.84	1.81	2.16	282	152
Q06	0.92	0.87	0.67	6052	830
Q07	0.84	0.76	0.71	712	182
Q12	1.13	1.07	1.00	5	6

1.33, 0.96) where the controlled measurement shows a win at every size, which is a larger error than any effect the table is meant to demonstrate.

8 Discussion

8.1 What these measurements support

A covering sized by cost over an equal-area integer key is a sound basis for a sky index, and the measurements identify precisely which queries it improves. For regions of $30'$ and larger it is faster than `pgSphere` warm and cold, at both catalogue sizes, and the margin grows with the catalogue – a factor 3.7 at 1° with cold caches at 50 million rows. For polygons and cross-matching it is faster by factors of 2.7 and 6–11. It does this with an index a third the size, built more than ten times faster, and with row estimates twice as accurate. The same conclusions hold on a density field taken from the real sky and on one designed by us, which is the strongest evidence we have that the cost model is not tuned to its test data.

8.2 What they do not

They do not establish that `skycell` is the better choice in general, and the 50-million-row run makes one failure sharper than the 10-million-row run did: for cones of an arcsecond to a few arcminutes the advantage shrinks with scale and reverses, so on a Gaia-sized catalogue the most common query of all – a small cone search on a warm index – is one this method makes slower, by 7–13%. On a 500 000-row ObsCore relation it is slower; on small cones over field-clustered observations it is slower at every size we measured; in a TAP service whose cost is dominated by protocol handling it changes nothing. Better row estimates did not, on the join shapes we wrote, produce better plans. And nothing here has run on real catalogue positions at Gaia density, on

more than one machine, or with more than one storage device.

8.3 Where the remaining cost is

Planning, and it gets worse rather than better with scale. A covering is computed per plan, and even cached it costs 0.015–0.13 ms against 0.009–0.011 ms for a GiST predicate that needs no such step. Worse, it is the one cost in the method that *grows* with source density, because a denser sky makes the model cut finer and emit more range arms: 0.019 to 0.031 ms at $6'$ between the two catalogue sizes, while `pgSphere`'s planning does not move. That is the whole of `skycell`'s disadvantage at small radii, and Sect. 6.3 shows it is what turns a 3% deficit at 10 million rows into a 13% deficit at 50 million.

The architectural way out is to stop planning ranges. An SP-GiST operator class over the cell hierarchy would present a single index qualification to the planner and refine during the index descent, where the work is proportional to the rows actually present rather than to the region, and would remove the per-arm planner cost that our measurements put at 15–20 μ s for each range in the disjunction. A cheaper intermediate is the = ANY single-scan form, which collapses the disjunction into one index scan at the price of a coarser covering.

The density synopsis is the other limit. An equi-depth histogram over a space-filling curve cannot see a cluster smaller than a bucket (Sect. 2.3); the area guard bounds the damage, but a custom `typanalyze` storing a multi-order density map – a MOC with counts – would estimate what the covering actually needs, and would improve selectivity estimates in crowded fields at the same time.

9 Conclusions

`skycell` keeps Q3C's compact integer key and B-tree while answering the region queries `pgSphere` was built for, by treating the depth of a covering as a cost question whose answer follows from the local source density and the price of an index range.

Under a protocol designed to remove the advantages that measurement order and cache state hand to whichever method is measured last, on two 10-million-row corpora – one drawn from the real Gaia DR3 density field – it is 6–11% faster than `pgSphere` for cone radii of $1''$ – $1'$ and 31–37% faster at 3° warm, and faster at every radius with caches dropped, by a factor of up to 2.5. It answers convex polygons 2.7–4.4 times faster, keeps an index a third the size of `pgSphere`'s – 38 GiB against 160 GiB at Gaia DR3 scale for the pair of indexes that covers the same ground, which is the difference between a 64 GiB server and a 256 GiB one – builds it thirty times faster, and estimates row counts twice as accurately. On cross-

matching, the operation Q3C exists for, it is Q3C’s equal (0.78–0.92 of its time, intervals reaching 1) and 2.5–3.3 times faster than pgSphere – so one index is competitive with each of the two established ones on the workload that each was built for. Its cost model’s one parameter is derived from the relation’s rows per page and the planner’s cost factors rather than configured – indistinguishable in speed from the constant it replaces, but no longer a number anyone has to choose – its chosen cell order costs 0.8–2% against the best available order, and its density estimate is unbiased wherever a density is defined.

It is not the better choice everywhere, and scale sharpens rather than softens that. Raising the catalogue to 50 million rows widens the advantage for regions of 30′ and above – to a factor 3.7 at 1° with cold caches – and reverses it below a few arcminutes, where `skycell` becomes 7–13% slower warm because planning a covering costs more in a denser sky while pgSphere’s planning does not. On a small or field-clustered relation it is slower for the same reason, and inside a TAP server whose cost is dominated by Python it changes nothing at all. The crossover measured on an ObsCore-shaped relation sits between 0.5 and 2 million rows for degree-scale cones and beyond 10 million for small cones on field-clustered data.

An index is worth choosing carefully when the scan dominates. Which queries those are is a property of the archive, and we have tried to say precisely which ones they were here.

Data availability

The extension, its test suites, the benchmark harness and the raw results of every measurement quoted here are available at <https://github.com/jesusjuansalgado/skycell> under the MIT licence. Both corpora are generated from seeds by the scripts recorded there – the Gaia corpus, at both scales, from the density map fetched from the ESA archive – so they are reproducible without transferring data; `bench/run.sh` and the numbered scripts 07–10 reproduce every table. The TAP A/B of Sect. 7 is recorded, protocol and results, in the same repository.

Acknowledgements. This work has made use of data from the European Space Agency (ESA) mission *Gaia* (<https://www.cosmos.esa.int/gaia>), processed by the *Gaia* Data Processing and Analysis Consortium (DPAC, <https://www.cosmos.esa.int/web/gaia/dpac/consortium>). Funding for the DPAC has been provided by national institutions, in particular the institutions participating in the *Gaia* Multilateral Agreement. Only per-cell source counts were used, to build the density field of Sect. 5.1.

The implementation, the benchmark harness and the analysis were developed with the assistance of Anthropic’s Claude Code. Because that bears on how the results should be read, we state what was done about

it: the geometric predicates and the covering are checked by the brute-force suites of Sect. 4, which compare them against independent computations over millions of adversarially placed points rather than against expected outputs; every indexed answer in the regression suites is required to equal a sequential scan with the exact predicate; and the error described in Sect. 2.4 was found by those tests rather than by inspection, which is the argument for relying on them. The benchmark harness was audited by reproducing its headline comparison under a different protocol (Sect. 6.4), which is how the difference between the two protocols came to light.

References

- Chilingarian, I., Bartunov, O., Richter, J., & Sigaev, T. 2004, in ASP Conf. Ser. 314, *Astronomical Data Analysis Software and Systems XIII*, ed. F. Ochsenbein, M. G. Allen, & D. Egret (San Francisco: ASP), 225
- Delli Veneri, M., & Salgado, J. 2026, *A&A*, submitted
- Demleitner, M., Neves, M. C., Rothmaier, F., & Wambsganss, J. 2014, *Astron. Comput.*, 7–8, 27
- Dowler, P., Rixon, G., Tody, D., & Demleitner, M. 2019, *IVOA Recommendation: Table Access Protocol Version 1.1*
- ESA 1997, *The Hipparcos and Tycho Catalogues*, ESA SP-1200
- Fernique, P., Boch, T., Donaldson, T., et al. 2014, *IVOA Recommendation: MOC – HEALPix Multi-Order Coverage map Version 1.0*
- Gaia Collaboration, Prusti, T., de Bruijne, J. H. J., et al. 2016, *A&A*, 595, A1
- Górski, K. M., Hivon, E., Banday, A. J., et al. 2005, *ApJ*, 622, 759
- Gray, J., Nieto-Santisteban, M. A., & Szalay, A. S. 2006, *The Zones Algorithm for Finding Points-Near-a-Point or Cross-Matching Spatial Datasets*, Microsoft Research Technical Report MSR-TR-2006-52
- Koposov, S., & Bartunov, O. 2006, in ASP Conf. Ser. 351, *Astronomical Data Analysis Software and Systems XV*, ed. C. Gabriel, C. Arviset, D. Ponz, & S. Enrique (San Francisco: ASP), 735
- Kunszt, P. Z., Szalay, A. S., & Thakar, A. R. 2001, in *Mining the Sky*, ed. A. J. Banday, S. Zaroubi, & M. Bartelmann (Berlin: Springer), 631
- Moon, B., Jagadish, H. V., Faloutsos, C., & Saltz, J. H. 2001, *IEEE Trans. Knowl. Data Eng.*, 13, 124

- Ortiz, I., Lusted, J., Dowler, P., et al. 2008, IVOA Recommendation: Astronomical Data Query Language Version 2.0
- Salgado, J., González-Núñez, J., Gutiérrez-Sánchez, R., et al. 2017, *Astron. Comput.*, 21, 22
- Salgado, J., et al. 2020, in ASP Conf. Ser. 522, *Astronomical Data Analysis Software and Systems XXVII*, ed. P. Ballester, J. Ibsen, M. Solar, & K. Shortridge (San Francisco: ASP), 307